



SWIGGY

Business context (Expedia-style)

- Customers browse restaurants, view menus, add items to cart, place orders, pay, restaurants accept & prepare, riders get assigned, pick up, and deliver. Revenue is primarily take rate on item totals plus platform fees, minus refunds/adjustments, with strong focus on on-time delivery, cancellations, and repeat behavior.

Tables & Relationships (Dimension Tables)

- **dim_date**(date_key {PK}, cal_date, year, quarter, quarter_name, month, month_name, week_of_year, day_of_month, day_of_week, day_name, is_weekend, is_month_start, is_month_end, is_quarter_start, is_quarter_end, is_year_start, is_year_end)
- **dim_time**(time_key {PK}, hour, minute, second, am_pm, hour_24, day_part)
- **dim_geo**(geo_sk {PK}, country_code, country_name, region_name, state_province, city, market_id, tz_name, latitude, longitude)
- **dim_customer**(customer_sk {PK}, customer_nk, first_name, last_name, email_hash, phone_hash, loyalty_tier, preferred_currency, preferred_lang, marketing_opt_in, segment, risk_flag, created_at, effective_from, effective_to, is_current) —> SCD2 Type
- **dim_device**(device_sk {PK}, device_type, os_name, os_version, app_web, app_version, browser_name, browser_version)
- **dim_channel**(channel_sk {PK}, source, medium, campaign, ad_group, ad_id, utm_term, utm_content, partner_id)
- **dim_restaurant** (restaurant_sk {PK}, restaurant_id_nk, restaurant_name, brand, cuisine_primary, rating, geo_sk {FK→dim_geo.geo_sk}, commission_band, is_chain_flag, open_time, close_time, effective_from, effective_to, is_current) —> SCD2 Type

Tables & Relationships (Dimension Tables)

- **dim_menu_item**(menu_item_sk {PK}, item_id_nk, restaurant_sk {FK→dim_restaurant.restaurant_sk}, item_name, category, veg_flag, spiciness_level, base_price, calories, allergens, effective_from, effective_to, is_current) —→ SCD2 Type
- **dim_delivery_partner**(rider_sk {PK}, rider_id_nk, full_name, phone_hash, vehicle_type, license_number, geo_sk {FK→dim_geo.geo_sk}, joining_date, effective_from, effective_to, is_current) —→ SCD2 Type
- **dim_payment_method**(payment_sk {PK}, payment_method, card_network, wallet_type, issuer_country, three_ds_enabled)
- **dim_promo**(promo_sk {PK}, promo_code, discount_type, discount_value, start_date_key {FK→dim_date.date_key}, end_date_key {FK→dim_date.date_key}, targeting_rules)
- **dim_fee_policy**(fee_policy_sk {PK}, policy_name, calculation_type, parameter_json, valid_from, valid_to)
- **dim_currency_rate**(currency_rate_sk {PK}, date_key {FK→dim_date.date_key}, currency_code, fx_to_report, fx_source, as_of_ts)

Tables & Relationships (Fact Tables)

- **fact_app_session**(session_id {PK}, date_key {FK→dim_date.date_key}, time_key {FK→dim_time.time_key}, customer_sk {FK→dim_customer.customer_sk}, device_sk {FK→dim_device.device_sk}, geo_sk {FK→dim_geo.geo_sk}, channel_sk {FK→dim_channel.channel_sk}, sessions, page_views, add_to_cart_events, checkout_events, created_ts)
- **fact_menu_impression**(impression_id {PK}, date_key {FK→dim_date.date_key}, time_key {FK→dim_time.time_key}, session_id {FK→fact_app_session.session_id}, channel_sk {FK→dim_channel.channel_sk}, restaurant_sk {FK→dim_restaurant.restaurant_sk}, menu_item_sk {FK→dim_menu_item.menu_item_sk}, impressions, price_shown, created_ts)
- **fact_add_to_cart**(add_event_id {PK}, date_key {FK→dim_date.date_key}, time_key {FK→dim_time.time_key}, session_id {FK→fact_app_session.session_id}, customer_sk {FK→dim_customer.customer_sk}, restaurant_sk {FK→dim_restaurant.restaurant_sk}, menu_item_sk {FK→dim_menu_item.menu_item_sk}, qty, item_price_atc, created_ts)

Tables & Relationships (Fact Tables)

- **fact_order_header**(order_id {PK}, date_key_ordered {FK→dim_date.date_key}, time_key_ordered {FK→dim_time.time_key}, customer_sk {FK→dim_customer.customer_sk}, device_sk {FK→dim_device.device_sk}, channel_sk {FK→dim_channel.channel_sk}, restaurant_sk {FK→dim_restaurant.restaurant_sk}, geo_sk {FK→dim_geo.geo_sk}, payment_sk {FK→dim_payment_method.payment_sk}, fee_policy_sk {FK→dim_fee_policy.fee_policy_sk}, promo_sk {FK→dim_promo.promo_sk}, gmv_amount, item_discount, restaurant_discount, platform_fee, delivery_fee, taxes, promised_dropoff_ts, actual_dropoff_ts, promised_ready_ts, ready_ts, take_rate_pct, status_code, created_ts)
- **fact_order_item**(order_id {FK→fact_order_header.order_id}, item_seq {PK}, menu_item_sk {FK→dim_menu_item.menu_item_sk}, qty, line_price, line_discount, addons_amount, veg_flag, category)
- **fact_assignment**(assignment_id {PK}, order_id {FK→fact_order_header.order_id}, restaurant_sk {FK→dim_restaurant.restaurant_sk}, offered_flag, accepted_flag, accept_ts)
- **fact_delivery_trip**(trip_id {PK}, order_id {FK→fact_order_header.order_id}, rider_sk {FK→dim_delivery_partner.rider_sk}, assigned_ts, to_pickup_start_ts, at_pickup_ts, to_dropoff_start_ts, dropped_ts, distance_km, duration_sec, surge_multiplier)
- **fact_cancellation**(cancellation_id {PK}, order_id {FK→fact_order_header.order_id}, date_key_cancel {FK→dim_date.date_key}, time_key_cancel {FK→dim_time.time_key}, cancel_party, reason_code, cancel_fee_amount, cancelled_ts)

Tables & Relationships (Fact Tables)

- **fact_payment**(payment_txn_id {PK}, order_id {FK→fact_order_header.order_id}, date_key {FK→dim_date.date_key}, time_key {FK→dim_time.time_key}, payment_sk {FK→dim_payment_method.payment_sk}, amount_authorized, amount_captured, amount_refunded, txn_type, payment_ts)
- **fact_refund**(refund_id {PK}, order_id {FK→fact_order_header.order_id}, date_key_refund {FK→dim_date.date_key}, time_key_refund {FK→dim_time.time_key}, refund_amount, chargeback_flag, reason_code, refund_ts)
- **fact_restaurant_availability**(availability_id {PK}, date_key {FK→dim_date.date_key}, restaurant_sk {FK→dim_restaurant.restaurant_sk}, open_minutes, outage_minutes, uptime_pct)
- **fact_rider_shift**(shift_id {PK}, rider_sk {FK→dim_delivery_partner.rider_sk}, date_key {FK→dim_date.date_key}, shift_duration_sec, active_duration_sec, idle_duration_sec)

Business Metrics & Respective SQL Queries

- 1) **App-to-Order Conversion Rate** -> % of unique sessions that result in ≥ 1 completed order in the period.

Formula: $\text{distinct orders} \div \text{distinct sessions} \times 100$

```
WITH s AS (  
  SELECT COUNT(DISTINCT session_id) AS sessions  
  FROM fact_app_session fs  
  JOIN dim_date d ON fs.date_key = d.date_key  
  WHERE d.cal_date BETWEEN :start_date AND :end_date  
)  
o AS (  
  SELECT COUNT(DISTINCT oh.order_id_nk) AS orders  
  FROM fact_order_header oh  
  JOIN dim_date d ON oh.date_key_ordered = d.date_key  
  WHERE d.cal_date BETWEEN :start_date AND :end_date  
  AND oh.status IN ('DELIVERED','PLACED','ACCEPTED') -- placed or better  
)  
SELECT  
  s.sessions,  
  o.orders,  
  ROUND(o.orders * 100.0 / NULLIF(s.sessions,0), 2) AS app_to_order_conversion_percent  
FROM s CROSS JOIN o;
```


Business Metrics & Respective SQL Queries

2) Average Order Value (AOV) -> Average pre-discount basket size for completed orders.

Formula: $\Sigma \text{ item_subtotal} \div \# \text{orders}$.

```
SELECT
  COUNT(*) AS orders,
  SUM(oh.gmv_amount) AS total_gmv,
  ROUND(SUM(oh.gmv_amount) / NULLIF(COUNT(*),0), 2) AS average_order_value
FROM fact_order_header oh
JOIN dim_date d ON oh.date_key_ordered = d.date_key
WHERE d.cal_date BETWEEN :start_date AND :end_date
AND oh.status = 'DELIVERED';
```

Business Metrics & Respective SQL Queries

3.) Platform Take Rate % -> Share of commercial revenue captured relative to GMV.

Formula: $(\sum (\text{item_subtotal} \times \text{take_rate}) + \sum \text{platform_fees} - \sum \text{refunds}) \div \sum \text{GMV} \times 100.$

```
WITH base AS (  
  SELECT  
    SUM(oh.gmv_amount) AS gmv,  
    SUM(oh.gmv_amount * oh.take_rate_pct + oh.platform_fee + oh.delivery_fee)  
      - COALESCE(SUM(rf.refund_amount), 0) AS net_revenue  
  FROM fact_order_header oh  
  LEFT JOIN fact_refund rf ON rf.order_id_nk = oh.order_id_nk  
  JOIN dim_date d ON oh.date_key_ordered = d.date_key  
  WHERE d.cal_date BETWEEN :start_date AND :end_date  
        AND oh.status IN ('DELIVERED')  
)  
SELECT  
  gmv,  
  net_revenue,  
  ROUND(net_revenue * 100.0 / NULLIF(gmv,0), 2) AS platform_take_rate_percent  
FROM base;
```

Business Metrics & Respective SQL Queries

4.) On-Time Delivery Rate (OTD) -> % of delivered orders where actual_dropoff_ts ≤ promised_dropoff_ts.

Formula: $\text{on_time_deliveries} \div \text{delivered_orders} \times 100$.

```
SELECT
  COUNT(*) AS delivered_orders,
  SUM(CASE WHEN oh.actual_dropoff_ts <= oh.promised_dropoff_ts THEN 1 ELSE 0 END) AS on_time_orders,
  ROUND(SUM(CASE WHEN oh.actual_dropoff_ts <= oh.promised_dropoff_ts THEN 1 ELSE 0 END) * 100.0
    / NULLIF(COUNT(*),0), 2) AS on_time_delivery_percent
FROM fact_order_header oh
JOIN dim_date d ON oh.date_key_ordered = d.date_key
WHERE d.cal_date BETWEEN :start_date AND :end_date
AND oh.status = 'DELIVERED';
```

Business Metrics & Respective SQL Queries

5.) **Order Cancellation Rate** -> % of placed orders that end in cancellation (restaurant, rider, or customer).

Formula: $\text{cancelled_orders} \div \text{placed_orders} \times 100$.

```
WITH placed AS (  
    SELECT COUNT(*) AS placed_orders  
    FROM fact_order_header oh  
    JOIN dim_date d ON oh.date_key_ordered = d.date_key  
    WHERE d.cal_date BETWEEN :start_date AND :end_date  
)  
cancelled AS (  
    SELECT COUNT(DISTINCT c.order_id_nk) AS cancelled_orders  
    FROM fact_cancellation c  
    JOIN dim_date d ON c.date_key_cancel = d.date_key  
    WHERE d.cal_date BETWEEN :start_date AND :end_date  
)  
SELECT  
    placed_orders,  
    cancelled_orders,  
    ROUND(cancelled_orders * 100.0 / NULLIF(placed_orders,0), 2) AS cancellation_rate_percent  
FROM placed CROSS JOIN cancelled;
```

Business Metrics & Respective SQL Queries

6.) Refund Rate -> % of delivered or cancelled orders that have any refund (full/partial).

Formula: $\text{refunded_orders} \div \text{placed_orders} \times 100$.

```
WITH placed AS (  
    SELECT COUNT(DISTINCT oh.order_id_nk) AS placed_orders  
    FROM fact_order_header oh  
    JOIN dim_date d ON oh.date_key_ordered = d.date_key  
    WHERE d.cal_date BETWEEN :start_date AND :end_date  
)  
ref AS (  
    SELECT COUNT(DISTINCT rf.order_id_nk) AS refunded_orders  
    FROM fact_refund rf  
    JOIN dim_date d ON rf.date_key_refund = d.date_key  
    WHERE d.cal_date BETWEEN :start_date AND :end_date  
)  
SELECT  
    placed_orders,  
    refunded_orders,  
    ROUND(refunded_orders * 100.0 / NULLIF(placed_orders,0), 2) AS refund_rate_percent  
FROM placed CROSS JOIN ref;
```

Business Metrics & Respective SQL Queries

7.) Repeat Customer Rate (90-day) -> % of customers with ≥ 2 completed orders in last 90 days.

Formula: $\text{customers}(\text{orders} \geq 2) \div \text{customers}(\text{with } \geq 1 \text{ order}) \times 100$.

```
WITH cust_orders AS (  
  SELECT customer_sk, COUNT(DISTINCT order_id_nk) AS orders_90d  
  FROM fact_order_header oh  
  JOIN dim_date d ON oh.date_key_ordered = d.date_key  
  WHERE d.cal_date BETWEEN date_sub(current_date(), 90) AND current_date()  
  AND oh.status = 'DELIVERED'  
  GROUP BY customer_sk  
)  
SELECT  
  COUNT(*) AS active_customers_90d,  
  SUM(CASE WHEN orders_90d >= 2 THEN 1 ELSE 0 END) AS repeat_customers_90d,  
  ROUND(SUM(CASE WHEN orders_90d >= 2 THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(*),0), 2)  
  AS repeat_customer_rate_percent  
FROM cust_orders;
```

Business Metrics & Respective SQL Queries

8.) Restaurant Acceptance Rate -> % of order offers accepted by restaurants.

Formula: $\text{accepted_offers} \div \text{total_offers} \times 100$.

```
SELECT
  r.restaurant_id_nk,
  r.brand,
  COUNT(*) AS total_offers,
  SUM(CASE WHEN a.accepted_flag = TRUE THEN 1 ELSE 0 END) AS accepted_offers,
  ROUND(SUM(CASE WHEN a.accepted_flag THEN 1 ELSE 0 END) * 100.0
    / NULLIF(COUNT(*),0), 2) AS acceptance_rate_percent
FROM fact_assignment a
JOIN dim_restaurant r ON a.restaurant_sk = r.restaurant_sk AND r.is_current = TRUE
JOIN dim_date d ON a.date_key = d.date_key
WHERE d.cal_date BETWEEN :start_date AND :end_date
GROUP BY r.restaurant_id_nk, r.brand
ORDER BY acceptance_rate_percent DESC;
```

Business Metrics & Respective SQL Queries

9.) Rider Utilization -> Share of rider shift time spent on “active” trip states (to-pickup, at-pickup, to-dropoff).

Formula: $\text{active_duration} \div \text{total_shift_duration} \times 100$.

```
SELECT
  COUNT(*) AS shifts,
  SUM(rs.shift_duration_sec) AS total_shift_sec,
  SUM(rs.active_duration_sec) AS total_active_sec,
  ROUND(SUM(rs.active_duration_sec) * 100.0 / NULLIF(SUM(rs.shift_duration_sec),0), 2)
    AS rider_utilization_percent
FROM fact_rider_shift rs
JOIN dim_date d ON rs.date_key = d.date_key
WHERE d.cal_date BETWEEN :start_date AND :end_date;
```


Business Metrics & Respective SQL Queries

10.) Order Preparation SLA Hit Rate -> % of orders where restaurant ready_ts ≤ promised_ready_ts.

Formula: $\text{prep_sla_hits} \div \text{accepted_orders} \times 100$.

```
SELECT
  COUNT(*) AS accepted_orders,
  SUM(CASE WHEN oh.ready_ts IS NOT NULL
    AND oh.ready_ts <= oh.promised_ready_ts
    THEN 1 ELSE 0 END) AS prep_sla_hits,
  ROUND(SUM(CASE WHEN oh.ready_ts IS NOT NULL
    AND oh.ready_ts <= oh.promised_ready_ts
    THEN 1 ELSE 0 END) * 100.0
    / NULLIF(COUNT(*),0), 2) AS prep_sla_hit_percent
FROM fact_order_header oh
JOIN dim_date d ON oh.date_key_ordered = d.date_key
WHERE d.cal_date BETWEEN :start_date AND :end_date
AND oh.status IN ('ACCEPTED','DELIVERED');
```